AI Chat History

Meeting Room Booking · prompts sent to the AI assistant while building the app

PROMPT 1

Full build prompt

```
Build a meeting room booking interface.

Stack: Next.js (App Router), TypeScript (strict), Route Handlers as the mock API
(in-memory store). Vitest for unit tests.

Working day: 09:00–18:00. The user can: pick a date, see existing bookings for
that date, choose start and end time, create, edit and delete a booking.

Business rules:
1. Start and end time must be within the working day.
2. start < end.
3. Minimum duration is 30 minutes.
4. Maximum duration is 2 hours.
5. Bookings cannot overlap.
   - Touching boundaries is not a conflict: 10:00–11:00 and 11:00–12:00 is
allowed.
6. Past time cannot be booked.
   - For today's date, past slots are unavailable.
   - For past dates, creating a booking is unavailable.
7. When editing, the current booking must not conflict with itself.
8. The server may return a conflict even if the UI considered the slot free.
   - This is expected behavior, not a bug.
   - The UI must handle it correctly: show a clear message, refresh the list of
bookings, and not lose the data the user entered.

API contract:
GET /api/bookings?date=YYYY-MM-DD
POST /api/bookings
PATCH /api/bookings/:id
DELETE /api/bookings/:id

type Booking = {
  id: string;
  date: string; // YYYY-MM-DD
  start: string; // HH:mm
  end: string; // HH:mm
  title?: string;
};

The server validates all business rules and returns 409 on conflict, 422 on
invalid data, and 404 on an unknown id.

Requirements:
```

- Keep pure domain logic (validation, overlap check) in its own folder, with no React or fetch. Share it between the form and the mock server.
- Use the date-fns library for date and time handling instead of custom code.
- Keep the network layer separate from the UI and easy to replace with a real API. Map HTTP statuses to typed errors (ConflictError, ValidationError, NotFoundError).
- Keep dates as YYYY-MM-DD strings (no timezone bugs).
- Cover these UI states: loading, empty, error with retry, and form submitting.
- On a 409: keep the form data, show a clear message, refetch the bookings, and re-validate against the fresh list.
- Add a demo switch to force a 409 (for example, a title containing "!conflict").
- Make the layout responsive and add basic accessibility: labels, aria-live messages, aria-invalid, visible focus.
- Write unit tests for edge cases: touching boundaries, edit excluding itself, exactly 30 min and 2 h, 08:30 start, 18:30 end, and past slots.
- Write a README covering how to run, what is done and not done, assumptions, and time spent.

Design is not evaluated; engineering quality is.

PROMPT 2

Success toast prompt

Add a success toast notification to the meeting room booking app (Next.js App Router, TypeScript).

Use the `sonner` library. Do not write a custom toast component.

Requirements:
1. Install `sonner` and render a single `<Toaster />` in `src/app/layout.tsx`. Use `position="top-right"`, `richColors`, and `closeButton`.
2. Show a success toast after a booking is created: "Booking created: 10:00–11:00 on 2026-10-29" (use the real start, end and date).
3. Show a success toast after a booking is updated: "Booking updated: 10:00–11:00".
4. Show a success toast after a booking is deleted: "Booking deleted".
5. Show an error toast when a request fails (network or server errors). Do not show an error toast for 409 conflicts. Keep the inline form message for those, because the form data must stay.
6. Trigger toasts from the UI layer only (in BookingApp.tsx and BookingForm.tsx). Keep src/api and src/domain free of any toast code.
7. Replace the existing "Saved." and "Booking deleted." text messages with toasts. Remove the now-unused `message` state.
8. Toasts must be accessible: the library's aria-live region stays enabled, and the toast must not steal focus from the form or the list.
9. Prevent duplicate toasts (for example, a double click on Submit) by disabling the submit button while saving and passing a stable `id` to each toast.

```
Change only what is needed for this feature. Do not refactor other code.
```

PROMPT 3

Select dropdown styling

Both time selects use the same pattern: `appearance-none`, a `relative` wrapper, a `ChevronDown` icon from lucide-react with `pointer-events-none`, and `peer` classes so the arrow follows focus and disabled state. Use identical class strings on Start and End so they look the same.